

```

import pandas as pd
import numpy as np

# Entropy calculation
def entropy(target_col):
    elements, counts = np.unique(target_col, return_counts=True)
    entropy = 0
    for i in range(len(elements)):
        prob = counts[i]/np.sum(counts)
        entropy += -prob * np.log2(prob)
    return entropy

# Information Gain
def info_gain(data, split_attribute, target_name="PlayTennis"):
    total_entropy = entropy(data[target_name])
    vals, counts = np.unique(data[split_attribute], return_counts=True)
    weighted_entropy = 0
    for i in range(len(vals)):
        subset = data[data[split_attribute] == vals[i]]
        weighted_entropy += (counts[i]/np.sum(counts)) * entropy(subset[target_name])
    return total_entropy - weighted_entropy

# ID3 Algorithm
def ID3(data, original_data, features, target_name="PlayTennis", parent_node_class=None):
    # If all target values are the same → return that value
    if len(np.unique(data[target_name])) <= 1:
        return np.unique(data[target_name])[0]
    # If dataset empty → return mode of original dataset
    elif len(data) == 0:
        return np.unique(original_data[target_name])[np.argmax(np.unique(original_data[target_name], return_counts=True)[1])]
    # If no features left → return parent node class
    elif len(features) == 0:
        return parent_node_class
    else:
        parent_node_class = np.unique(data[target_name])[np.argmax(np.unique(data[target_name], return_counts=True)[1])]
        # Choose best feature
        gains = [info_gain(data, feature, target_name) for feature in features]
        best_feature = features[np.argmax(gains)]
        tree = {best_feature: {}}
        # Grow tree

```

```

        for value in np.unique(data[best_feature]):
            sub_data = data[data[best_feature] == value]
            subtree = ID3(sub_data, original_data, [f for f in features if f != best_feature], target_name, parent_node_class)
            tree[best_feature][value] = subtree
        return tree

# Prediction
def predict(query, tree, default="Yes"):
    for key in list(query.keys()):
        if key in tree.keys():
            try:
                result = tree[key][query[key]]
            except:
                return default
            if isinstance(result, dict):
                return predict(query, result)
            else:
                return result
    return default

# -----
# DEMO RUN
# -----
dataset = pd.DataFrame([
    ["Sunny", "Hot", "High", "Weak", "No"],
    ["Sunny", "Hot", "High", "Strong", "No"],
    ["Overcast", "Hot", "High", "Weak", "Yes"],
    ["Rain", "Mild", "High", "Weak", "Yes"],
    ["Rain", "Cool", "Normal", "Weak", "Yes"],
    ["Rain", "Cool", "Normal", "Strong", "No"],
    ["Overcast", "Cool", "Normal", "Strong", "Yes"],
    ["Sunny", "Mild", "High", "Weak", "No"],
    ["Sunny", "Cool", "Normal", "Weak", "Yes"],
    ["Rain", "Mild", "Normal", "Weak", "Yes"],
    ["Sunny", "Mild", "Normal", "Strong", "Yes"],
    ["Overcast", "Mild", "High", "Strong", "Yes"],
    ["Overcast", "Hot", "Normal", "Weak", "Yes"],
    ["Rain", "Mild", "High", "Strong", "No"]
], columns=["Outlook", "Temperature", "Humidity", "Wind", "PlayTennis"])

features = dataset.columns[:-1]
tree = ID3(dataset, dataset, features)

```

```
print("Decision Tree:", tree)

# Classify a new sample
query = {"Outlook": "Sunny", "Temperature": "Cool", "Humidity": "High", "Wind": "Strong"}
print("Prediction for new sample:", predict(query, tree))
```

Decision Tree: {'Outlook': {'Overcast': 'Yes', 'Rain': {'Wind': {'Strong': 'No', 'Weak': 'Yes'}}}, 'Sunny': {'Humidity': {'High': 'No', 'Normal': 'Yes'}}},
Prediction for new sample: No